

Functional Specification

Perfect

Caolan William Cochrane: 22490802

Ethan Doyle: 22497082

Date Finished: 27/11/2025

0. Table of Contents

0. Table of Contents	2
1. Introduction	3
1.1. Overview	3
1.2. Business Context	3
1.3. Glossary	3
2. General Description	4
2.1. Product / System Functions	4
2.2. User Characteristics and Objectives	5
2.3. Operational Scenarios	5
2.4. Constraints	5
3. Functional Requirements	7
3.1. 3D Rendering and Camera Control	7
3.2. Multi-User Collaborative Editing	7
3.3. Object Manipulation	8
3.4. Real-time Scene Validation	8
3.5. Model Importing	9
3.6. Project Saving and Loading	9
3.7. Scene Representation & Scene Creation	9
4. System Architecture	11
5. High-Level Design	12
6. Preliminary Schedule	13
7. Appendices	14

1. Introduction

1.1. Overview

Perfect is an online desktop application that allows interior designers to create a 3D space and interact in real time. It will enable these designers and users to collaboratively design and join each other in a shared environment. As well as this, there will be functionality for scene manipulation, allowing users to visualise interior layouts interactively from any machine.

Most interior designers today use tools that are only made for a single user in mind. They often have to send screenshots back and forth to get feedback which can cause poor communication to get in the way and can lead to confusion or lost ideas. Perfect acts as a way to streamline this process, offering a shared environment that makes communication and co-operative experimentation easier.

The project is written in C++, with graphics and rendering being handled by OpenGL and SDL3.

1.2. Business Context

Not applicable. There are plans to ask interior designers for their thoughts on our project (and to see if anything could be improved), but other than that there will be zero business context.

1.3. Glossary

- **OpenGL:**
A widely used programming interface for rendering 3D and 2D graphics using vectors.
- **glTF:**
A royalty-free specification for the efficient transmission and loading of 3D scenes and models by engines and applications. glTF minimizes the size of 3D assets, and the runtime processing needed to unpack and use them. [1]
- **tinyGLTF:**
A C++ header only library for glTF. Helps to parse glTF files.

- **SDL3:**
A cross-platform development library designed to provide low level access to audio, keyboard, mouse, joystick and graphics hardware.
- **Scene Validation:**
Makes certain that nothing erroneous is occurring within a 3D scene. For example, objects clipping into one another.
- **Replication:**
Creating and maintaining multiple copies of data / resources across different systems to ensure consistency.

2. General Description

2.1. Product / System Functions

The intention of this project is to provide interior designers with an online desktop application where they can design in a collaborative space.

The major functionality of our project is as follows:

- **Rendering Engine**
 - The application will feature a rendering engine, built with OpenGL, that will handle and support lighting, materials, textures and geometric transformations.
- **Scene Management**
 - The application will contain scenes which contain the hierarchy of transforms and other spatial data related to any models (or any other miscellaneous objects) the interior designers may be using and/or making.
 - Users will have the ability to save their 3D environments. Importing and exporting will also be a feature, which will be possible through glTF and tinyGLTF.
 - Scenes will also contain scene validation - there will be working physics so objects cannot collide and clip into each other.
- **Models & Object Manipulation**
 - The application will feature methods to upload and edit models, including ones found from the web. This is achieved through glTF and tinyGLTF.
 - For editing purposes, there will be basic methods like moving, rotating, scaling and colouring interior elements. Users will also be able to edit properties of the model, like their transform, or spatial data directly.
 - Models will also have full texture support, courtesy of glTF.
 - As stated previously, objects & models will have a modicum of physics, just enough so that they do not clip inside of each other. There will be nothing like gravity, etc. in the project, as it's unnecessary.
- **User Interface & Interactivity**
 - The application will feature a user interface with intuitive camera controls, snapping, and simple menus for adding and modifying objects. The UI will also feature the hierarchy of whatever scene a user may be working in, as mentioned previously.

- There will also be a graphical user interface, built with ImGui, which will be how our users will join and interact with scenes.
- **Real-Time Collaboration**
 - Users will be able to collaborate in real time in the application, as the scene will update to show other users's changes. This entails synchronization of the scene state between multiple users which will allow multi-user editing and communication.
 - There will also be a chat in the user interface to facilitate better communication.

Additionally, here is a list of our main functional requirements, which will be elaborated on in section 3.

2.2. User Characteristics and Objectives

As stated, we foresee that our main users will be interior designers: they are who we are making the application for as they are our main target audience. They will have knowledge about interior design and its process. Additionally, they will have many creative & technical skills, as well as general knowledge involving design principles, materials and construction knowledge. Most professional designers use software in 2025, so we also expect a familiarity with design software.

To meet the expectations of our main users, our application will provide realistic rendering alongside intuitive 3D modelling. This is a basic necessity for any design software. Lastly, we also expect our real-time collaboration features to be a very desirable characteristic for our target audience - interior designers will be expecting a way to communicate and work with their teams.

2.3. Operational Scenarios

- **Creating a new Project:**
 - The user would launch the application and choose to start a new project after being presented with the main dashboard. They would be prompted to name it, and perhaps some details like project dimensions, et cetera. The application would then generate a blank 3D space which the user can see and navigate around + prepare to place models.
 - The system would display tool panels for object insertion, model import, object manipulation and more.
- **Saving and Loading a Project**
 - After working for any amount of time, the user clicks save project. The system saves all geometry details, object placements, spatial data,

imported assets and any other scene-related data/info. The application saves the project and takes the user to the dashboard / home screen. The user then opts to close the application. Any amount of time later, they re-open the application and re-launch the project. All objects will appear in the same location and the system will restore the previous camera view and position.

- **Importing a 3D Model**

- The user wants to import a 3D model they found on the internet of, say, a couch. They download the glTF file for the couch from wherever they found it, and click “import model” in the application. The application prompts the user to select the file from their computer’s files. The user then will be prompted to confirm the import, and then the application will display a small preview and add it to the asset library and scene automatically. If the model comes with textures, the model will also automatically have their base textures applied.

- **Inviting Other Users to Edit**

- The user wants to add other users as editors for the project. After launching the project they can select an option to invite a team member. Once the team member has joined, the interface will update to show that another collaborator is present. If the first user were to move a sofa, the second user would also see the sofa moving, and would see it shift on their screen. The second user can adjust the lighting and the first user would see it shift immediately. Both users continue to modify different areas of the project.

- **Modifying an Object**

- The user wants to modify an object already present in the scene. After clicking on the object, the interface will update to show details of the object, i.e. its transform coordinates, colour details, texture details and so on. The user can also see arrows on the object which they can drag. If they drag on an arrow, the object will be stretched in the direction the user is dragging, essentially modifying the shape of the object. The user can also edit the object’s position by dragging the object around by the center, or by modifying its position coordinates in the details menu.

2.4. Constraints

- **Hardware constraints**

- This is not exactly an exhaustive constraint, and is more dependent on the software we are using. OpenGL uses the GPU of the user’s

hardware for rendering, so if either of us has a poor computer, it could affect our testing, or the performance of the application.

- It could also affect the potential users of our application, that being interior designers.
- However, this is not a critical error, as most hardware nowadays has functional enough GPUs for the purposes of this application. Additionally, on the users end, interior designers would be expected to have good hardware to meet their design needs. It is mostly on our end and for testing purposes where it could cause problems.
- In case of worse hardware, however, we will be building the application with poorer hardware in mind.

- **Model constraints**

- Currently, our application only supports models of the glTF format. We do not have any plans currently to increase support for other model file formats. This means that the application will be limited in terms of models - if one wants to reuse a model, it must be available in the glTF file format.
- Luckily, glTF is one of the most popular file formats and has become a standard for transmitting 3D models. We do not expect this to be a huge issue in the future.

- **Time constraints**

- There is a chance our scope is too ambitious and we may need to redo our scope later.

3. Functional Requirements

3.1. 3D Rendering and Camera Control

- **Description:**
 - The system shall, at a baseline, render objects in a 3D space, alongside a camera to view objects in the space. The camera will be able to move at the user's behest, with quick, intuitive movement. The system shall achieve this through OpenGL.
- **Criticality:**
 - Very high - this is the very baseline of the project. We cannot do anything else without this.
- **Technical issues:**
 - Requires a rendering loop.
 - Must implement a camera model.
 - Needs to handle window resizing.
- **Dependencies with other requirements:**
 - Provides the visual interface (window) needed by most requirements to do with rendering, i.e. scene creation, object manipulation and so forth.

3.2. Multi-User Collaborative Editing (Replication)

- **Description:**
 - The system shall allow multiple users to edit the same project concurrently. Each user's actions (i.e. adding, deleting, or transforming objects) should be reflected in all connected clients in real time. Essentially, client side interactions and server side interactions must be synchronized across all users in real-time.
- **Criticality:**
 - Very high - real time collaboration is a core aspect of our project. Without this, our system would be a single-user editor.
- **Technical issues:**
 - Requires a client-server architecture or equivalent server layer to broadcast all changes to each client.
 - Must have a data model for scenes that can be synchronized.
 - We have to handle concurrent objects to the same object.
 - Network latency can cause issues (for example, packet loss and/or lag).
- **Dependencies with other requirements:**

- It will depend on object manipulation and scene validation as it defines what will be broadcasted to other clients.

3.3. Object Manipulation

- **Description:**
 - The system shall allow users to select objects in the 3D scene and apply transformations onto the objects. These transformations range from rotation, scaling to position changes. The resulting transform must be visually reflected in the object in real time.
- **Criticality:**
 - Very high, to meet the basic requirements of a design software. Basic object placement and adjustment is a core design philosophy.
- **Technical issues:**
 - Transforms must be applied in a consistent manner.
 - We need a method to determine which object has been selected (for example, ray casting through camera).
- **Dependencies with other requirements:**
 - Interacts with real time collaboration as changes must be broadcast to other clients.
 - Depends on 3D rendering and camera control so that objects can be seen and selected.

3.4. Real-time Scene Validation

- **Description:**
 - The system shall perform basic scene validation checks such as detecting overlapping models/objects when objects are placed or moved. It will automatically prevent these from happening and notify the user of this occurring.
- **Criticality:**
 - Medium. The system functions fine already without scene validation, but it supports the goal of interior designers being able to produce usable layouts.
- **Technical issues:**
 - Requires geometric calculations as a part of physics to detect for any potential collisions/intersections.
 - Will have to use regularly used physics methods such as bounding boxes.
 - Need to have an efficient strategy to not cause a performance overhead.

- **Dependencies with other requirements:**
 - Depends on object representation and spatial data from object manipulation since an intersection may occur after transformations.

3.5. Model Importing

- **Description:**
 - The system shall allow users to import 3D models, whether external or not. The system will only allow supported file formats, in this case, glTF. The system will also support exporting (more on this later).
- **Criticality:**
 - High. It is not required to demonstrate collaboration, but many interior designers use external models - it is important to support them.
- **Technical issues:**
 - Requires parsing one or more 3D file formats.
 - Needs to handle missing textures.
- **Dependencies with other requirements:**
 - Depends on 3D rendering to render the imported model with OpenGL.

3.6. Project Saving and Loading

- **Description:**
 - The system shall allow users to save the current state of their project, as well as loading pre-existing projects. The project will save the entire scene hierarchy as well as any objects (alongside their designated info) & spatial data inside said hierarchy.
- **Criticality:**
 - High. There needs to be a source of persistent storage.
- **Technical issues:**
 - Requires a scene serialization format which will be capable of representing all scene data.
 - Needs to handle file I/O errors.
- **Dependencies with other requirements:**
 - Depends on most requirements to do with the scenes themselves.
 - Interacts with collaboration if saving occurs while collaboration is occurring.

3.7. Scene Representation & Scene Creation

- **Description:**

- The system shall allow users to create 3D scenes which will be stored in a hierarchy and store any info related to objects & their transformations & displaying the objects to the camera.
- **Criticality:**
 - High, as this is where the majority of user work will be happening.
- **Technical issues:**
 - Need to store multiple scenes with differing data in a hierarchy.
- **Dependencies with other requirements:**
 - Depends on any requirement that stores spatial data and/or data related to objects, such as object manipulation.
 - Interacts with scene validation, as validation happens within scenes.

4. System Architecture

4.1. Reused Components

- **OpenGL:**
 - We use OpenGL as our rendering API to draw scenes and to render objects and models. OpenGL is the most widely used rendering API.
- **SDL3:**
 - SDL3 is a cross-platform development library designed to provide low level access to audio, keyboard, mouse, joystick and graphics hardware.
 - We use SDL3 to handle window creation, event handling and cross-platform I/O. The event handling is mostly to do with player input, i.e. keyboard and/or mouse input.
- **glTF and tinyglTF:**
 - glTF is a royalty-free specification for the efficient transmission and loading of 3D scenes and models by engines and applications, and tinyglTF is a C++ header only file for glTF that helps to parse models.
 - We use glTF and tinyglTF to allow users to load and import 3D models in our project.

4.2. Architecture Overview

Our system follows a modular client-server architecture. The client application handles rendering, scene editing, user input and local state management whereas the server will handle multi-user collaboration, project storage and replication + synchronization.

The architecture is organized into major system modules, which are shown below alongside details about their main functionality.

4.2.1. Project Module (Saving and Loading)

- Handles everything to do with projects associated with users - saving and loading their data, and keeping track of the data itself and making sure everything is consistent.

- Serializes most of the data within a project, such as spatial data, object transforms, the objects themselves, position data, and so on.
- Interacts with the scene module as the scene module is where most of the data that the project module stores is - the project module needs to store the state of all the scenes in the project.

4.2.2. Rendering Module

- Responsible for drawing the 3D scene, objects, models, lighting and UI overlays using OpenGL.
- Handles shaders, buffers, textures and camera matrices.
- Depends on SDL3 for creating the OpenGL context as well as windows to view the context in.

4.2.3. User Input Module

- Responsible for capturing user mouse and keyboard input frame by frame using SDL3.
- Input is used for many different modules & events, like moving the camera, moving an object, placing an object, dragging to move, clicking “accept” buttons, et cetera.
- Inputs can be passed to many other modules depending on what the input is used for, i.e. dragging to move an object will pass the input to the scene manipulation module.

4.2.4. Scene Module

- Scenes are where objects, transforms, meshes, materials, lighting and so on are stored and contained.
- Responsible for maintaining the internal representation of the aforementioned objects, transforms etc.
- Handles multiple scenes, but only one scene is active at once. Each project can contain multiple scenes, but users are only on one scene at a time.
- Interacts with scene validation as it needs to know how to resolve collisions, etc. on scene state changes.

4.2.5. Networking and Synchronization Module

- Handles synchronization between several clients and the server.

- Broadcasts edits to other users in real time.
- Handles receiving these updates and applying them to the scene of other users' clients.

4.2.6. Import and Export Module

- Handles the importing and exporting of 3D models of the glTF format.
- Converts files into a readable format through tinyglTF.
- Adds these model assets into the application's project library.

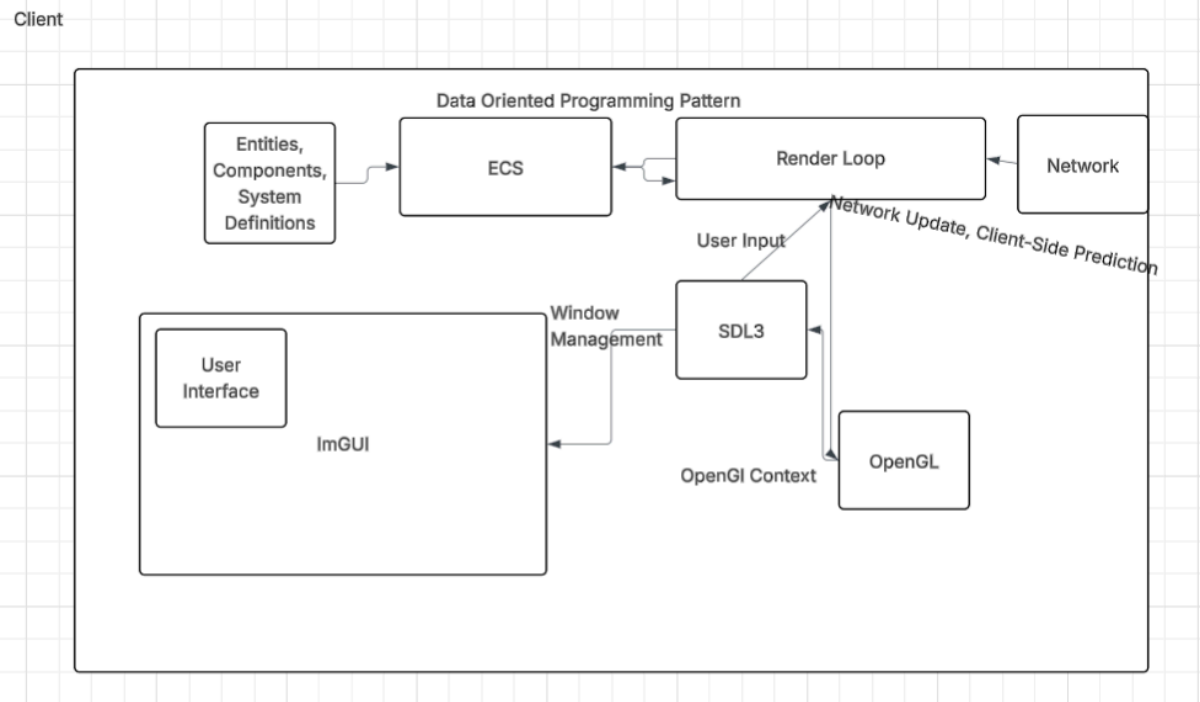
4.2.7. Validation Module

- Handles scene validation (collision detection and other rule-based checks).
- Reports validation issues to the UI and flags objects/areas that fail validation.
- Interacts with the scene module to gather data from any objects that are failing validation checks.

4.2.8. UI

- The UI of the application, provides menu systems, panels, toolbars and so on.
- Displays collaboration status, validation messages alongside general object + model details.
- Enables the user to interact with the system (choosing to edit, change object details, et cetera).

5. High-Level Design

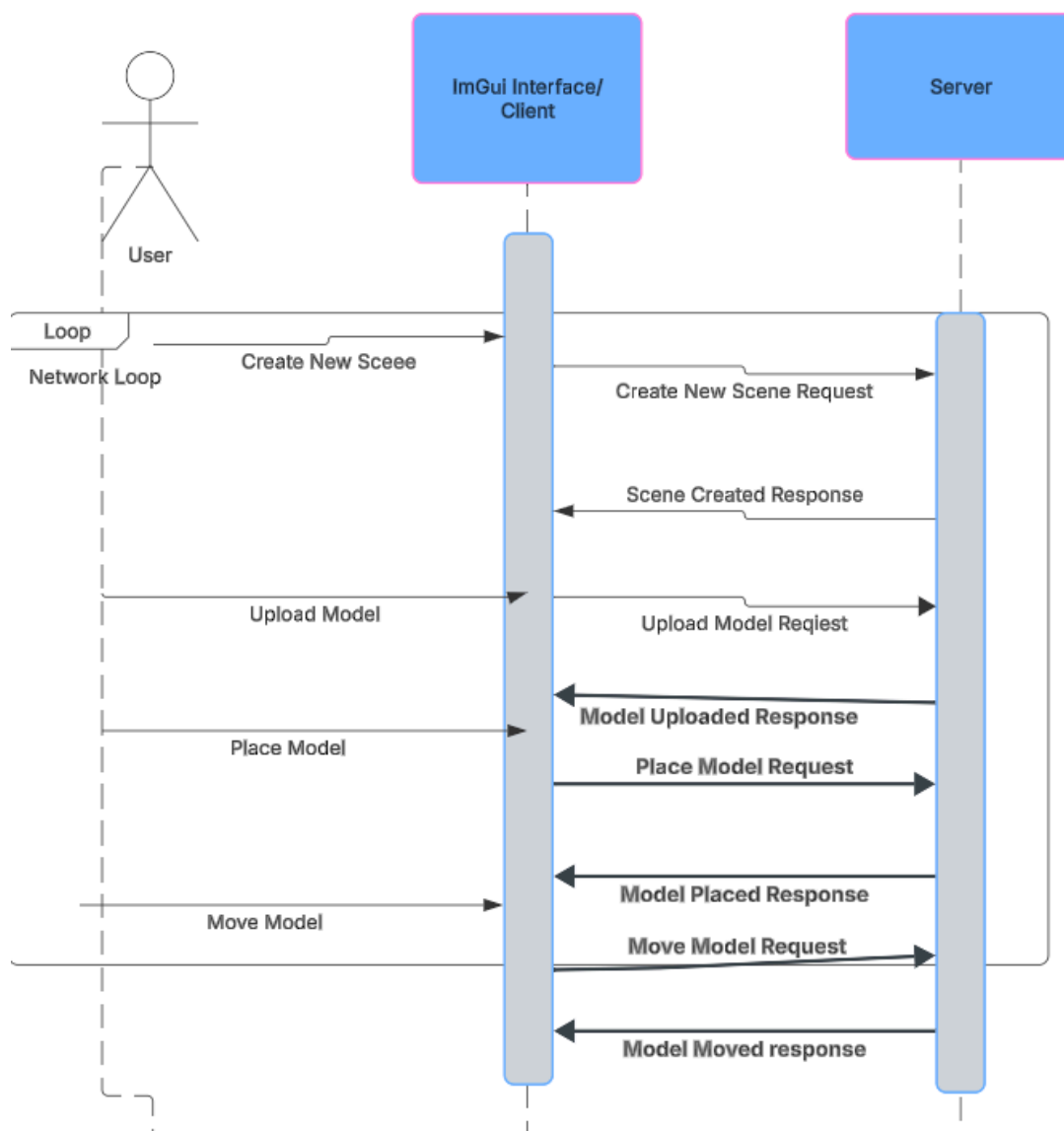


This diagram shows how we structure the client side of *Perfect* and how all the main pieces fit together when the application is running. We mainly use it to highlight how input, rendering, ECS logic, networking and the UI all link up. It also helps show how we keep things separate so the client stays responsive.

On the client, we use an ECS (Entity Component System) to store all of our scene data, the components, and any systems we need for updating things. The render loop runs every frame and asks the ECS for whatever it needs to draw the scene using OpenGL. SDL3 looks after input and window management for us, and ImGui gives us the UI that interior designers actually interact with. Our networking layer is responsible for sending user actions to the server and updating the client's local state when the server sends updates back. Since we're using a data-oriented style, we try to keep each subsystem as simple and self-contained as possible.

In terms of flow, the user interacts with the UI or camera and SDL3 picks up the events. These get passed into the ECS or UI, depending on what's needed. The render loop then takes whatever the ECS currently holds and pushes it to OpenGL, which draws the frame. Meanwhile, the networking layer handles outgoing requests (like model placement) and incoming authoritative updates from the server. All of this means the client can feel smooth and responsive even when there's a lot happening.

We went with this design because it keeps everything nicely separated. Rendering doesn't get tangled with simulation, input doesn't get tangled with networking, and ECS logic stays consistent and predictable.



5.2. Client–Server Interaction Sequence

This sequence diagram shows what happens when a user performs some of the main actions in the system, like creating a new scene, uploading a model, placing it, or moving it. It basically shows the “conversation” between the client and the server inside the network loop.

When the user clicks something in the ImGui interface—like “Create New Scene”—the client forms a request and pushes it into the networking loop. This request gets sent to the server, which checks it, processes it, and updates its ECS with whatever change is needed. The server then sends a response back, and the client updates its own ECS to match. The render loop then picks up the new data and redraws the scene. The same pattern repeats for uploading a model, placing it into the world, and moving it around the scene.

We follow this pattern because it keeps things simple and predictable. The client stays smooth and fast since it only has to send requests and apply updates, while the server handles all the heavy lifting of validation and replication.

6. Preliminary Schedule

6.1. GANTT Chart & Project Plan

Task Name	Dec 2025	Jan 2026	Feb 2026	Mar 2026
Networking Implementation (Multi Users)				
Replication				
Scene Logic				
Object Manipulation Features				
Project Persistence (Saving and Loading, etc.)				
Basic UI				
Scene Validation				
Debugging and Testing				

- We have already implemented basic rendering, a 3D viewport, basic model loading & textures so our focus from here on out will be on the real-time collaboration aspect of the project. These include a basic networking implementation and basic replication.
- We expect these features to take a while. We will be starting somewhere in the middle to end of December due to exams, and will likely finish around the first half of January.
- After then, we will begin working on our scene logic (only one scene active at once) and serializing information that appears in scenes.
- We will then begin working on object manipulation features around the same time as scene logic, and after these features are finished we will begin working on project persistence, towards the end of January.
- Project persistence involves manufacturing projects and serializing information that can be saved, making sure projects appear the same when closed and re-opened. We expect this one to take a while as well, and may take most of February to complete.
- After that, it is mostly additional features, like basic UI and scene validation. These are still important to the project however, as we need a basic UI - it's just not our priority as we can still have a functioning project without it.
- To end off, we will be conducting debugging and testing throughout the project's lifespan, but we will dedicate ourselves to debugging and testing especially towards the end of the lifespan, around February when project persistence is being worked on.

6.2. Minimum Viable Product

- For our MVP, we want to at least have the following features:
 - A working UI.

- Networked users (multiplayer, synchronization).
 - Object placement.
 - Replication.
 - Ability to load models on two different machines.
- These features are in line with our GANNT chart - we focus on the networking & replication aspects and then do object manipulation features afterwards.

7. Appendices

7.1 References

[1] Khronos Group. “glTF: The 3D Asset Delivery Format.” [Online]. Khronos.
Available: <https://www.khronos.org/glTF/>